

Game Engines – Part 1/2



Image sources: <https://jackbrummet.blogspot.de/2007/10/video-games-why-waste-good-technology.html>

What is a Game? (1/2)



- Video Games are Soft Real-Time Simulations
 - Scientific: “soft real-time interactive agent-based computer simulations”
- Computer Simulation
 - A subset of a (imaginary) world is modeled mathematically so that it can be manipulated by a computer.
 - The model is an approximation to and a simplification of (an imaginary) reality, because it is impractical to include every detail, like atoms or quarks.
 - Approximation and simplification are two of the game developer's most powerful tools!
- Agent-based simulation
 - A simulation in which number of distinct entities (agents) interact.
 - Due to this nature, object-oriented languages are very well suited to create agent-based simulations.

What is a Game? (2/2)



- Interactive real-time simulations
 - The state changes over time (dynamic)
 - Unpredictable player inputs (interactivity)
 - Real-time response and feedback
- Soft real-time systems
 - A system in which missed deadlines are not catastrophic.
 - If the frame-rate dies, the human player generally doesn't!
- Hard real-time systems
 - Any missed deadline can have severe consequences.
 - Possibly even the death of the human operator!
 - The avionics system in a helicopter
 - The control rod system in a nuclear power plant

=> Games are soft real-time systems!

Deadlines



- Games have numerous deadlines to meet
 - Rendering (~30 frames / second)
 - Physics (~120 updates / second)
 - Artificial Intelligence (~think once per second)
 - Audio Library (~60 updates / seconds)
 - ... and many more!

=> Different lengths and priorities!

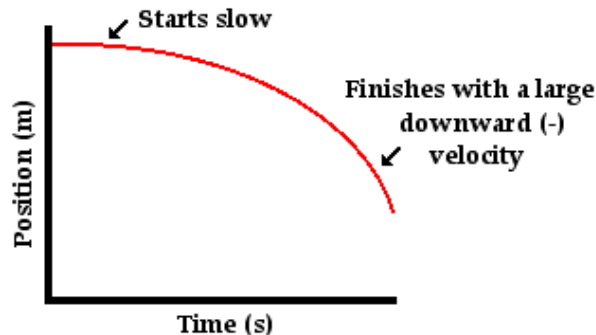
- How are these deadlines typically prioritized?
 - High: Audio / Physics
 - Lower: Rendering
 - Lowest: AI

Mathematical models (1/2)



- Analytic model (closed-form)
 - Can be evaluated any value of its independent variables
 - Example: Rigid body falling under the influence of gravity

$$y(t) = \frac{1}{2} g t^2 + v_0 t + y_0$$



- Very convenient, when such a model can be found...
 - Many mathematical problems don't have a closed-form solution
 - Player input is unpredictable
- We can not hope to model the entire game analytically

Image sources: <http://www.physicsclassroom.com/class/1DKin/Lesson-5/Representing-Free-Fall-by-Graphs>

Mathematical models (2/2)



- Numerical model

- Same example: Rigid body falling under the influence of gravity

$$y(t + \Delta t) = f(y(t), \dot{y}(t), \ddot{y}(t), \dots)$$

- The height at some future time $(t + \Delta t)$ can be found as a function of the height and its first and second time derivatives at the current time t .
 - Typically implemented by running calculations repeatedly
 - Determine the systems state at each discrete time step
 - Games work this way!
 - A “Game Loop” runs repeatedly
 - Each game system updates its state for the next discrete time step
 - Results are then “rendered”
 - Graphics, Sound and other output

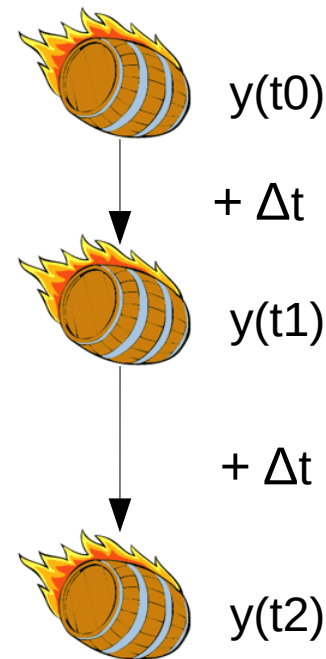


Image sources: <http://momgeekedition.blogspot.de/>

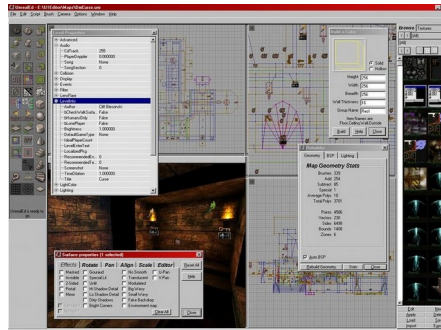
What is a Game Engine? (1/2)



- The term Game Engine arose in the mid 1990s
 - id Software's Doom was architected with a reasonable separation between its core components and its game specific components
 - Developers began re-tooling games into new products
- In the late 1990s games like Unreal and Quake III Arena were designed with modding in mind
 - Highly customizable engines with scripting languages (e.g. Quake C)
 - Engine licensing became viable secondary revenue stream for developers



Doom (1993)



UnrealEd 1.0 (1998)



Quake III Arena (1999)

Image sources: <http://doom.wikia.com/wiki/Imp>, https://en.wikipedia.org/wiki/Quake_III_Arena, <https://en.wikipedia.org/wiki/Unreal>

What is a Game Engine? (2/2)



- Today, most developers license game engines to create their games
 - Reuse of significant proportions of key software components
 - Still a considerable investment in custom software engineering
 - Usually much more economical than developing everything in-house
 - Examples of modern game engines



Fortnite



Deponia (PS4, iOS)



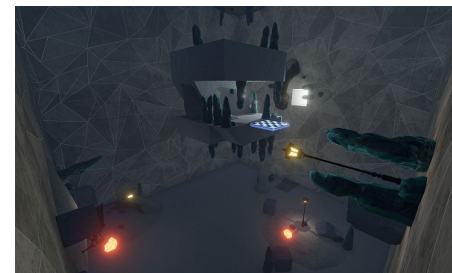
Outer Wilds

Image sources: <https://cdn2.unrealengine.com/26br-keyart-noshadow-1920x1080-3358aa3ee425.jpg>, <https://fr.flossmanuals.net/godot-game-engine/game-examples/>, https://en.wikipedia.org/wiki/Outer_Wilds

Engine vs. Game



- The line between a game and its engine is often blurry
 - Towards the end of production, the line usually gets even blurrier
- Arguably, a data-driven architecture is what makes the difference
 - Game engines should be extensible, so that they can be used to create many different types of games
- Re-usability gamut
 - There will always be a trade-off between optimality and generality



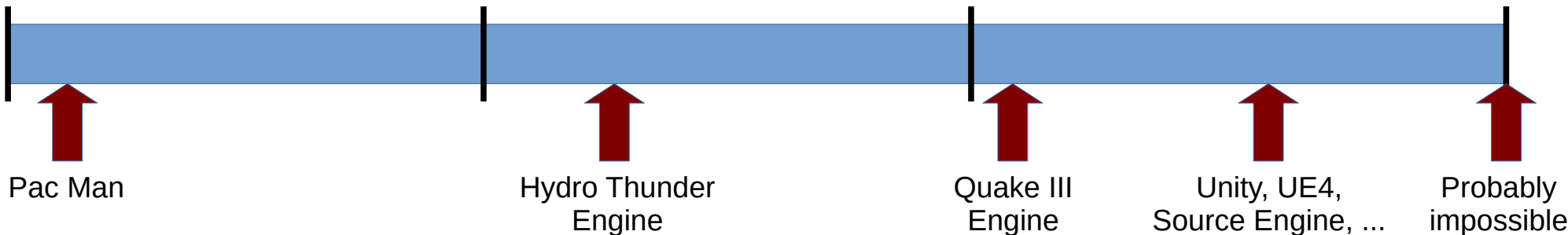
Gravity Cub3d

Cannot be used to build more than one game

Can be customized to make very similar games

Can be *modded* to build any game in a specific genre

Can be used to build any game imaginable



Offline and Runtime Components



- A Game Engine generally consists of...
 - Offline components (aka tool suite)
 - Runtime components

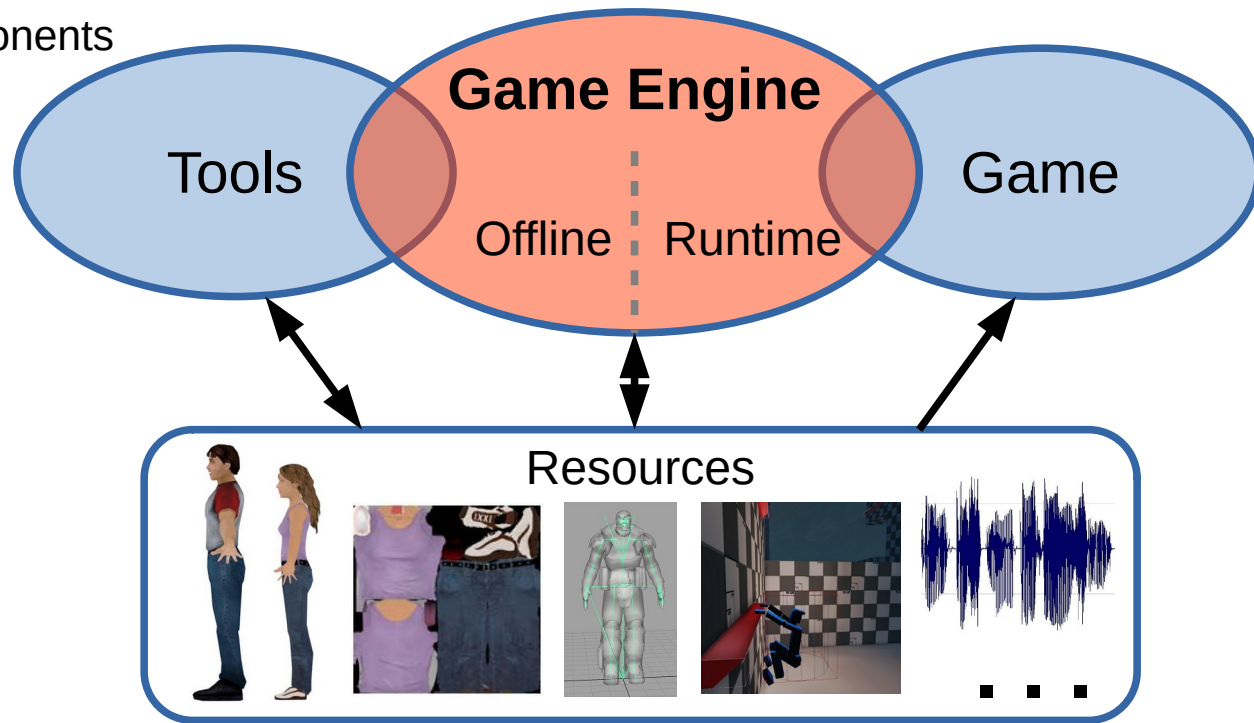


Image sources: <https://udn.epicgames.com>, <http://www.froyok.fr/blog/2012-02-exil-dev-blog-1-physics-in-udk-and-colossus>, <http://captivatingound.com/audio-only-menus/>

Runtime Engine Architecture (1/2)



- Game engines are built in layers
- Upper layers depend on lower layers
- Circular dependencies should be avoided

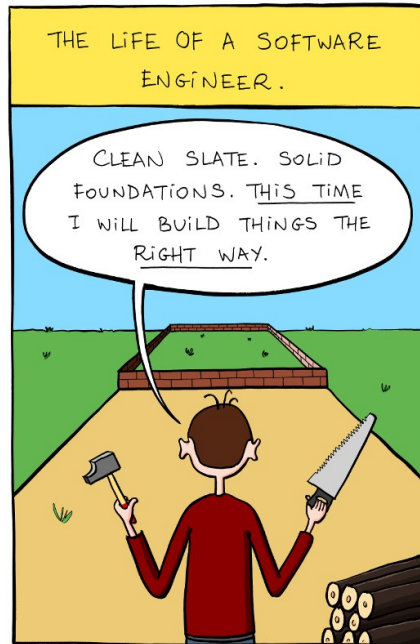
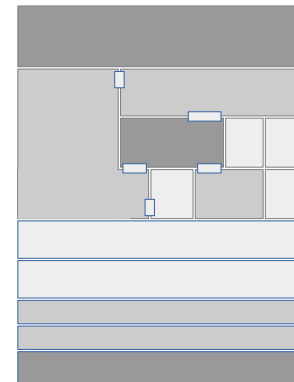


Image sources: <http://dev.mixrad.io/blog/2014/02/24/Bringing-MixRadio-to-the-new-Nokia-X-family>

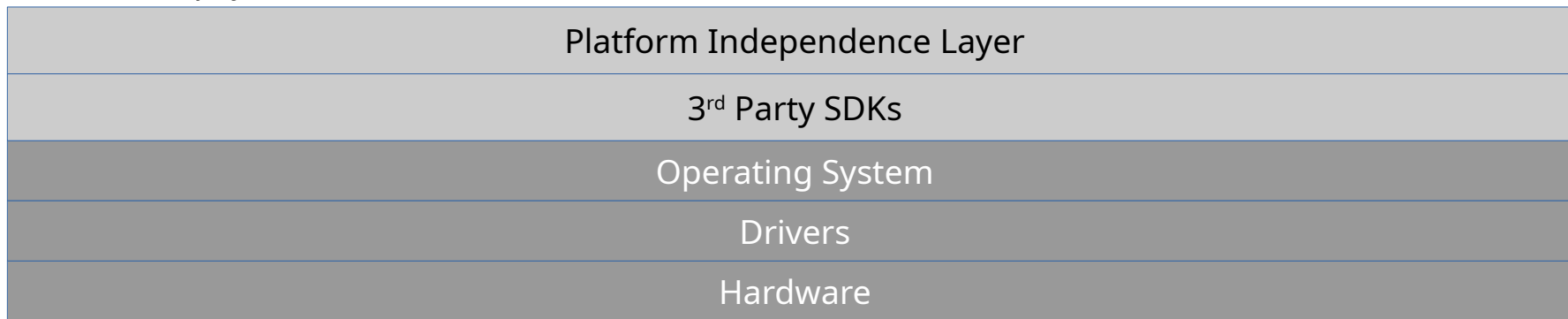
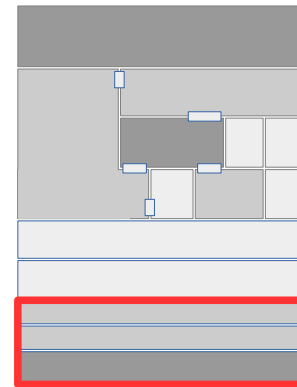


Theory of Game Development

Hardware and Platform Layers



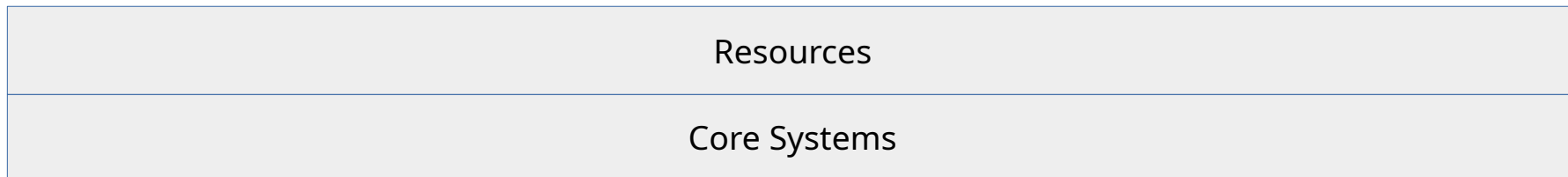
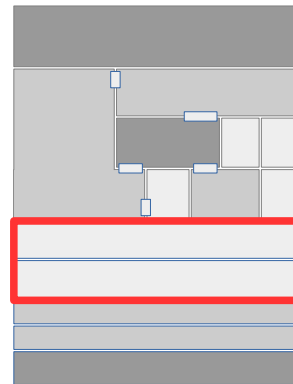
- 3rd Party SDKs
 - Graphics (DirectX, OpenGL, Vulkan, LibGCM, ...)
 - Data structures and algorithms (STL, Boost, ...)
 - Also physics, character animation, AI, sound, and so on
- Platform Independence Layer
 - Shields the rest of the engine from the knowledge of the platform
 - Typically wraps around...
 - Platform detection, data types, file system, network code, hi-res timer, threading, graphics, physics, ...



Core Systems



- Core Systems
 - Assertions
 - Memory management
 - Math library (vectors, matrices, quaternions, ...)
 - Custom data structures and algorithms
- Resources
 - Resource manager (eventually with a unified interface)
 - Typically distinct types for every resource with dedicated loaders
 - Mesh resource, texture resource, shader resource, sound resource, ...
 - Advanced functionalities such as streaming or hot-reloading



High-Level Systems (1/3)

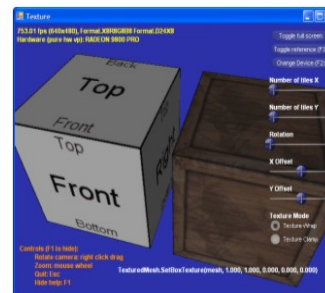
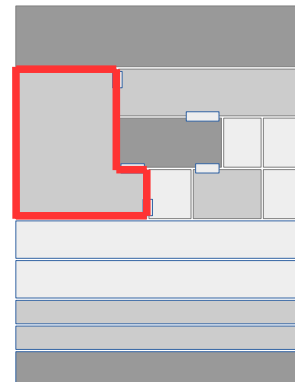
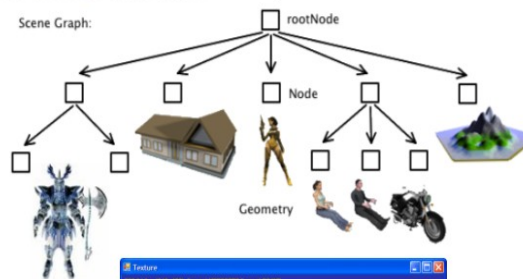
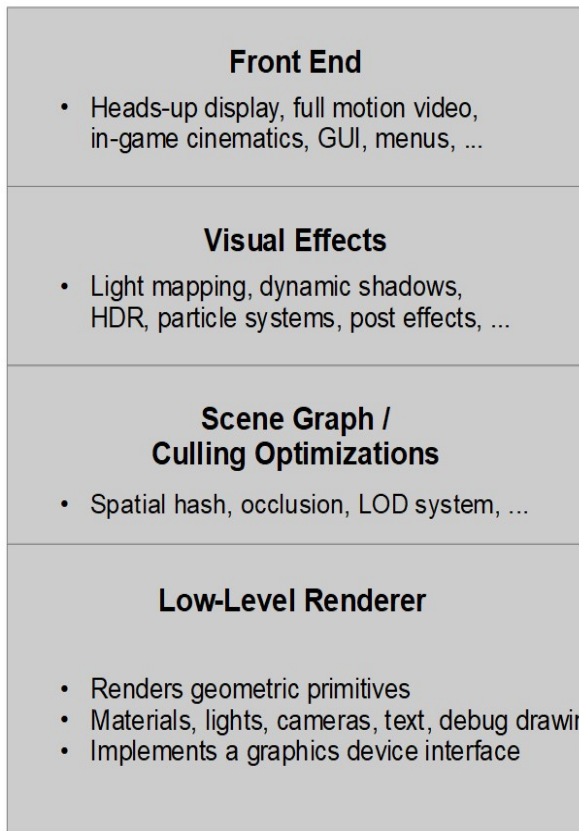


Image sources: <http://dev.mixrad.io/blog/2014/02/24/Bringing-MixRadio-to-the-new-Nokia-X-family>, <http://wiki.jmonkeyengine.org/doku.php/jme3:advanced:spatial>, <http://www.bit-tech.net/gaming/pc/2011/07/08/crysis-2-directx-11-patch-analysis/4>

High-Level Systems (2/3)

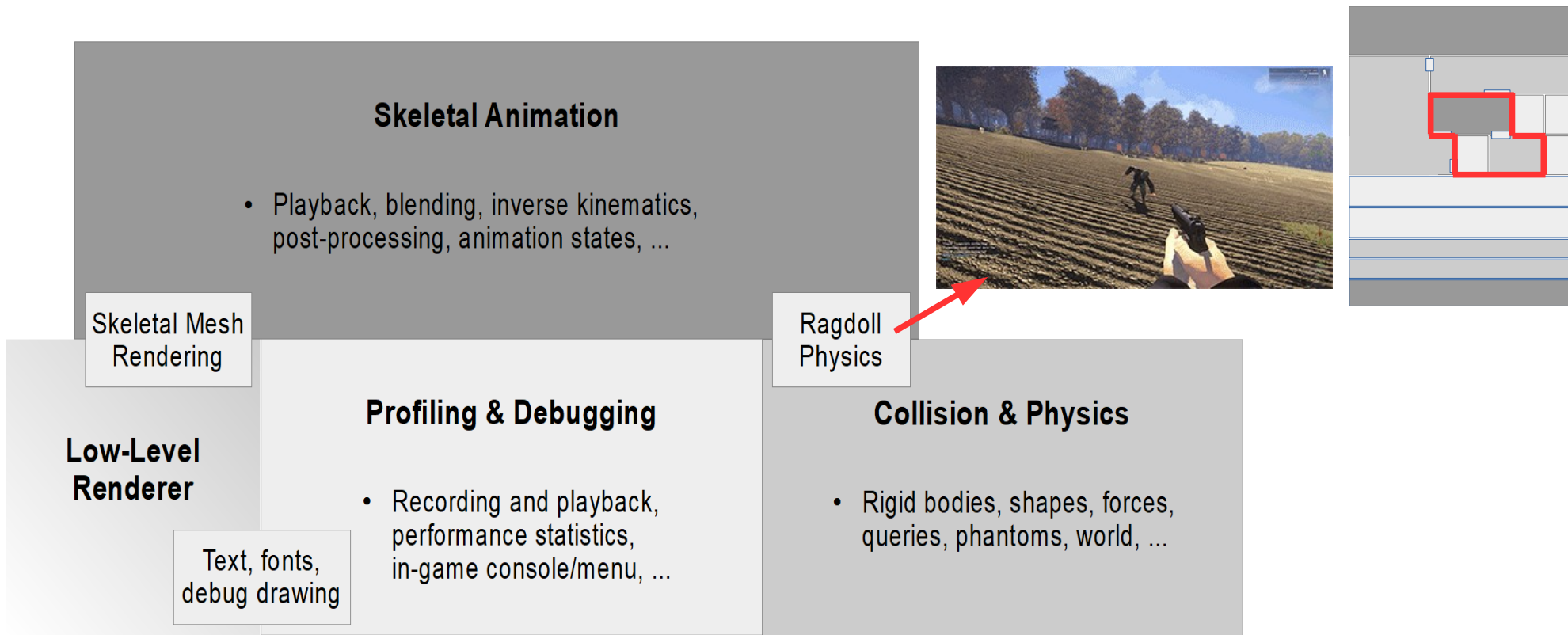


Image sources: http://www.reddit.com/r/dayz/comments/1ii799/thats_why_the_standalone_really_needs_ragdoll

High-Level Systems (3/3)



Online Multiplayer • Match-making • Object authority policy • Game state replication • ...	Audio • Audio playback and management • DSP and effects • 3D sound model • SFX, speech, music, atmosphere
	Human Interface Devices (HID) • Keyboard, mouse, game controllers • Custom input mapping • Chords, sequences, gestures • Input and output (rumble, LED, ...)

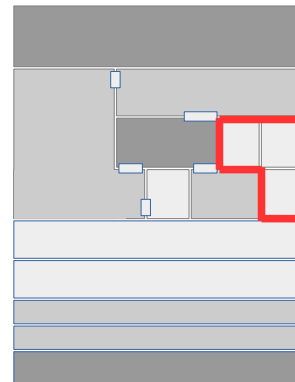


Image sources: www.gamerinvestments.com, www.zopomobileshop.com, www.theprimacy.com, www.amazon.com, www.circuitsathome.com, www.microsoftstore.com, www.pixshark.com

Gameplay Systems (1/2)



- Gameplay Foundations
 - Game Worlds and Object Models
 - Scripting and Event System
 - High-Level Game Flow System (e.g. FSM)
 - Artificial Intelligence Foundations

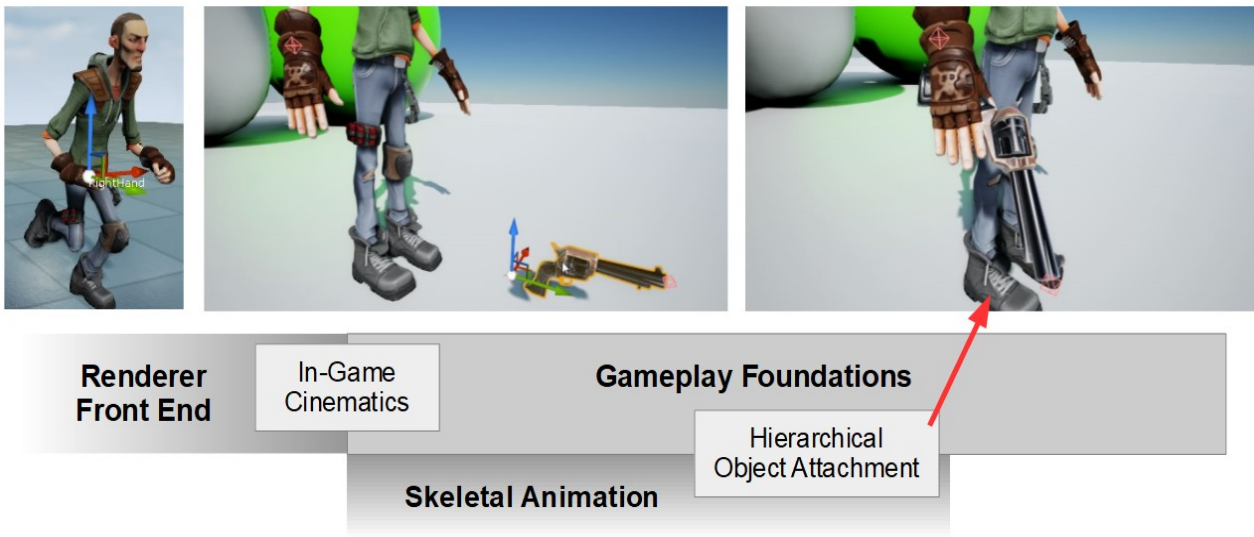
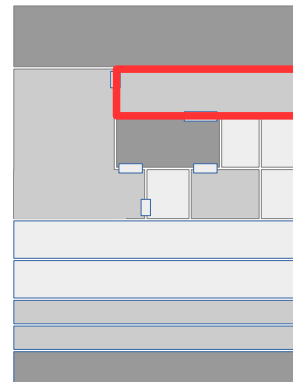
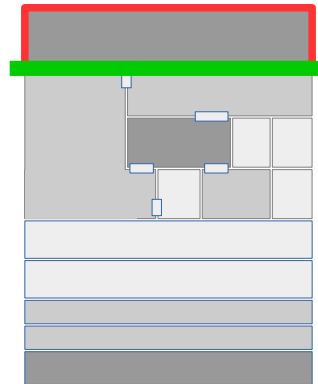


Image sources: <https://docs.unrealengine.com/latest/INT/Engine/Content/Types/SkeletalMeshes/Socket/index.html>

Gameplay Systems (2/2)



- Game Specific Subsystems
 - Player mechanics
 - Game cameras
 - Artificial intelligence
 - Game-specific rendering
 - Weapons, vehicles, puzzles, power-ups, quests, enemies, ...



=> If a clear line could be drawn between the engine and the game, it would be below this layer!

Game Specific Subsystems

Tools and the Asset Pipeline



- Digital Content Creation (DCC)
 - Used to create a game engine's input data
 - Output data is rarely suitable for direct in-game use
- The Asset Conditioning Pipeline (ACP) typically involves three stages
 - 1) Exporters
 - 2) Resource compilers
 - 3) Resource linkers
- The Resource Database typically stores...
 - all resources and their interdependencies
 - the metadata needed for the ACP

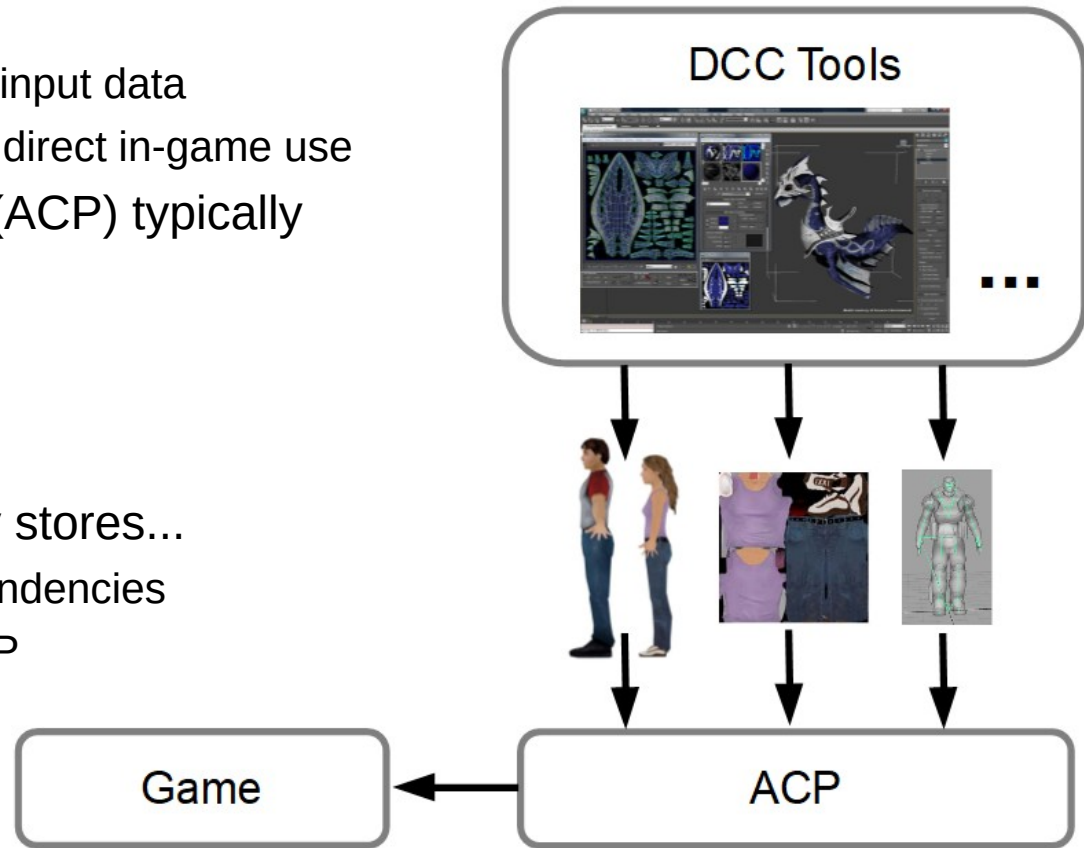
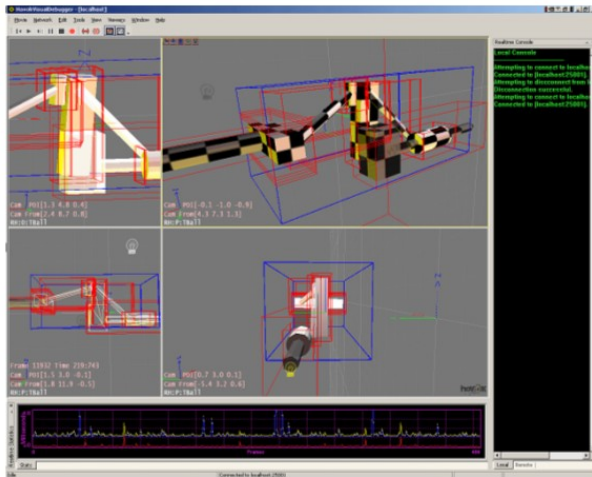
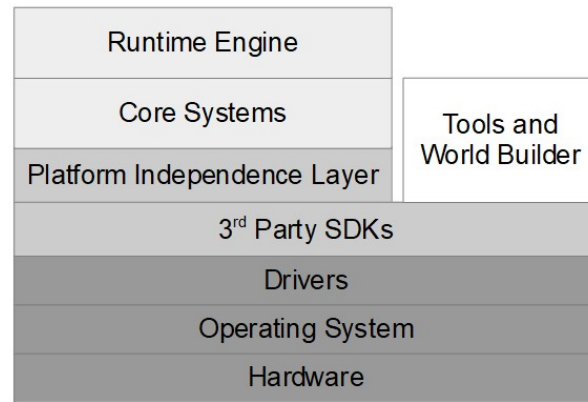


Image sources: <https://udn.epicgames.com>, <http://www.autodesk.com>

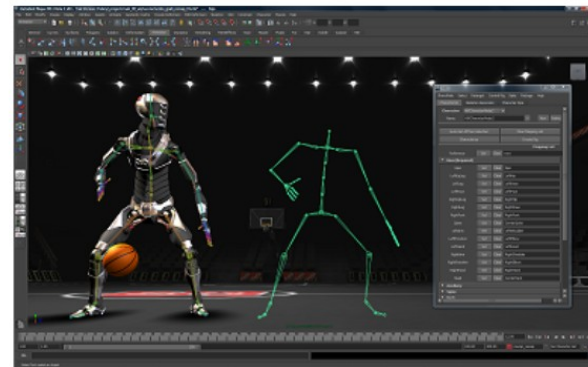
Tool Architecture (1/3)



- Stand alone tools architecture
- The tools are built independently from the game engine
- The engine may provide plugins
- Possible communication via TCP/IP



Havok Visual Debugger



Autodesk Maya

Image sources: http://anarchy.cn/manual/12/HavokSdk_ProgrammersManual/bk02ch07s03.html, <http://www.gamedev.net/topic/653352-blender-for-making-games>

Tool Architecture (2/3)



- Tools built on a framework shared with the game
- Ideally, the tools can then run on the development PC and on the target platform
- This is especially useful for console development
- Short turn around times
- Representative results

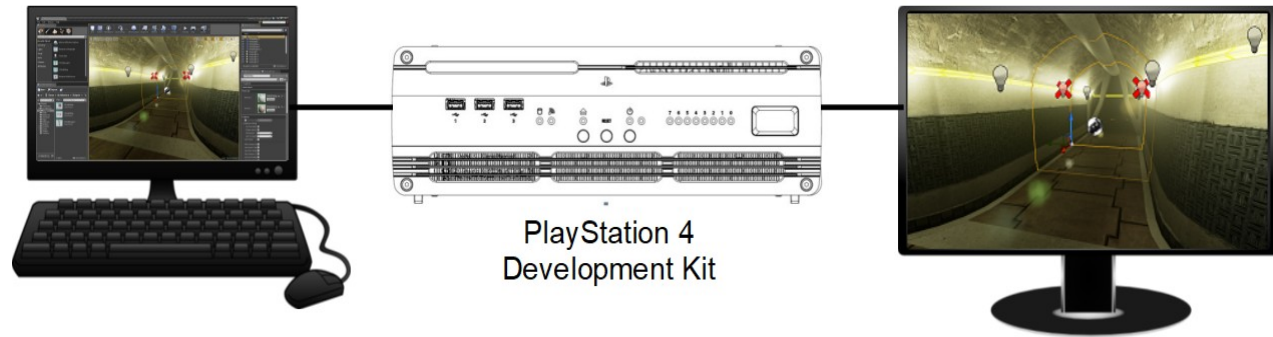
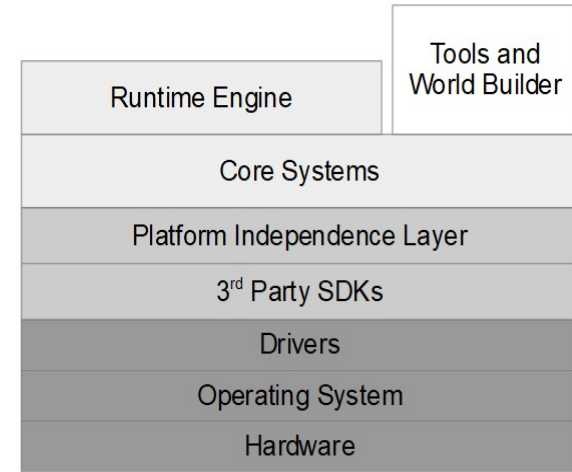
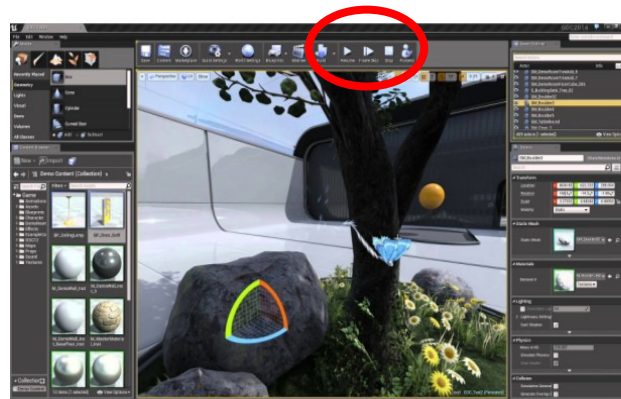
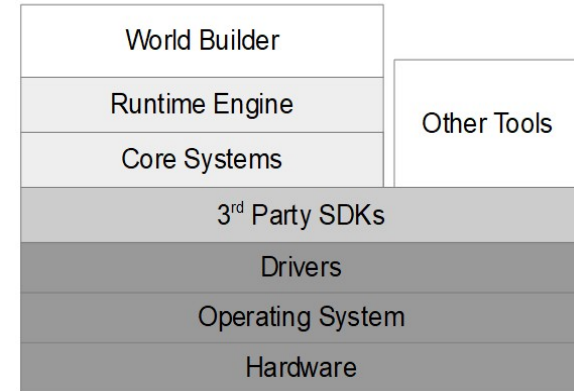


Image sources: <http://pixshark.com/computer-desktop-screen-clipart.htm>, <http://www.engadget.com/2013/07/16/sony-ps4-development-kit-fcc>, <http://caveingame.com/udk-vs-unreal-engine-4>

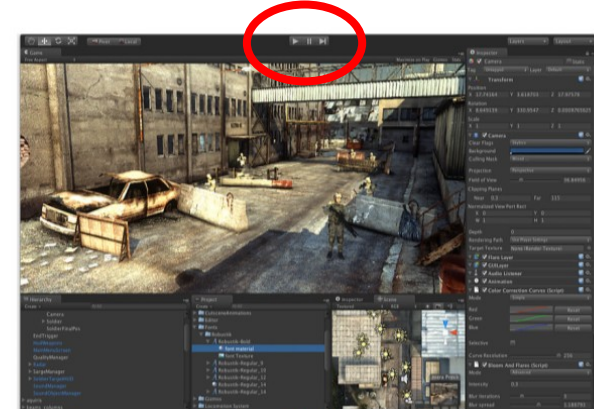
Tool Architecture (3/3)



- Tools built into the game itself
- The tools have access to the full range of the data structures used by the game engine
- Only one representation is needed
- Running the game from within the editor is very fast



Unreal Engine 4 Editor



Unity 5 Editor

Image sources: <https://www.youtube.com/channel/UCBobmJyysJ6Ll7UbfiI4iwQ>, <http://www.giantbomb.com/unity-engine/3015-5683/>

Conclusions



- Video Games are Soft Real-Time Simulations
 - Missed deadlines are not catastrophic
- Game Engines...
 - are extensible systems with a data-driven architecture
 - make multi-platform development much easier
 - comprise offline and runtime components
 - are architected in multiple layers
 - are extremely huge and complex software systems
 - allow to manage complex asset pipelines
 - utilize various different tool architectures